Nombre: Kevin Vaca
Arquitectura: MISD

```
#include <iostream>
#include <chrono>
#include <thread>

#include <cmath>
#include <vector>

#include <fstream>
#include <sstream>
#include <iomanip>

int main () {
    // Archivo
    std::ifstream archivo ("/mnt/garage/code/computacion-paralela/05_examen/datos.csv");
    if (!archivo.is_open()) {
        std::cout<<"Error: No se puede abrir datos.csv" << std::endl;
        return 1;
    }
    // Vectores
    std::vector <int> id;
    std::vector <std::string> vivienda;
    std::vector <int> lectura_anterior;
    std::vector <int> lectura_actual;
    std::vector <double> tarifa;
    std::vector <int> dias;
    // Encabezado
    std::string linea;
    getline (archivo, linea);
    // Llenar Vectores
    while(getline(archivo, linea)) {
        if(linea.empty()) {
            continue;
        }
        std::stringstream sss (linea);
        // Recoger
        std::string datoId;
        std::string datoVivienda;
        std::string datoAnterior;
        std::string datoActual;
        std::string datoTarifa;
        std::string datoDias;

        getline(sss, datoId, ',');
        getline(sss, datoVivienda, ',');
```

```
    getline(sss, datoAnterior, ',');
    getline(sss, datoActual, ',');
    getline(sss, datoTarifa, ',');
    getline(sss, datoDias, ',');

    // Guardar
    id.push_back(std::stoi(datoId));
    vivienda.push_back(datoVivienda);
    lectura_anterior.push_back(std::stoi(datoAnterior));
    lectura_actual.push_back(std::stoi(datoActual));
    tarifa.push_back(std::stod(datoTarifa));
    dias.push_back(std::stoi(datoDias));
}
archivo.close();

std::cout<<"Dataset cargado."<<std::endl;
std::cout<<"Total de viviendas: "<< id.size() << std::endl;


int total = static_cast<int>(id.size());
if (total != 100) {
    std::cout<<"ERROR no se cargo los elementos del DATASET" << std::endl;
    return 1;
}

// Vectores para Resultados
std::vector <double> consumo_electrico(total, 0.0);
std::vector <double> costo(total, 0.0);
std::vector <double> promedio_diario(total, 0.0);


// Tiempo
auto empezar = std::chrono::high_resolution_clock::now();

std::thread h1([&]() {
    for (int i = 0; i < total; i++) {
        consumo_electrico[i] = lectura_actual[i] - lectura_anterior[i];
    }
}
);

std::thread h2([&]() {
    for (int i = 0; i < total; i++) {
        costo[i] = (lectura_actual[i] - lectura_anterior[i]) * tarifa[i];
    }
}
);
```

```
    std::thread h3([&]() {
        for (int i = 0; i < total; i++) {
            // Mutiplicado por 1.0 para que se obtengan los decimales
            promedio_diario[i] = (lectura_actual[i] - lectura_anterior[i]) * 1.0 / dias[i];
        }
    }
    );

    h1.join();
    h2.join();
    h3.join();

    auto finalizar = std::chrono::high_resolution_clock::now();

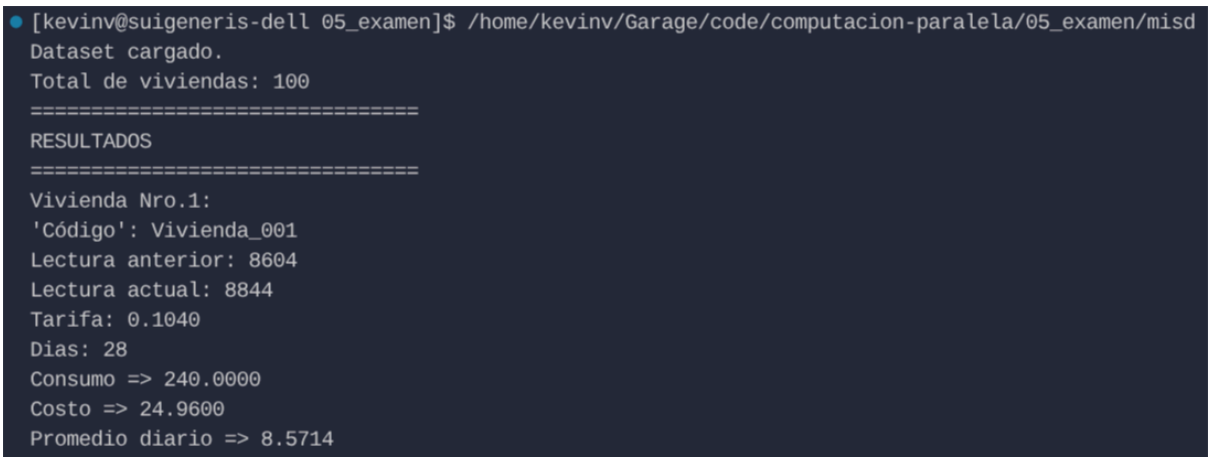
    std::chrono::duration <double, std::micro> tiempo_total = finalizar - empezar;

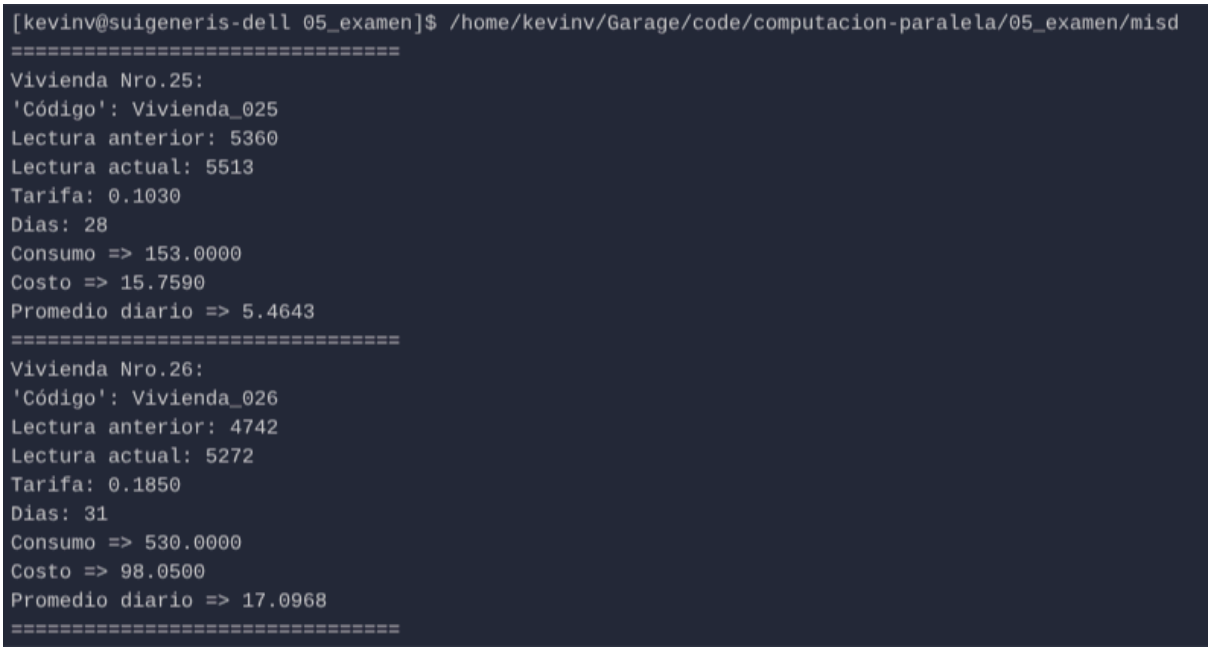
    std::cout << std::fixed << std::setprecision(4);

    std::cout<<"================================"<<std::endl;
    std::cout<< "RESULTADOS "<< std::endl;
    std::cout<<"================================"<<std::endl;

    for (int i = 0; i < total; i++) {
        std::cout<<"Vivienda Nro."<<id[i]<<":"<<std::endl;
        std::cout<<"'Código': "<<vivienda[i]<<std::endl;
        std::cout<<"Lectura anterior: "<<lectura_anterior[i]<<std::endl;
        std::cout<<"Lectura actual: "<<lectura_actual[i]<<std::endl;
        std::cout<<"Tarifa: "<<tarifa[i]<<std::endl;
        std::cout<<"Dias: "<<dias[i]<<std::endl;
        std::cout<<"Consumo => "<<consumo_electrico[i]<<std::endl;
        std::cout<<"Costo => "<<costo[i]<<std::endl;
        std::cout<<"Promedio diario => "<<promedio_diario[i]<<std::endl;
        std::cout<<"================================"<<std::endl;

    }
    std::cout<<"Tiempo de Ejecución Total: "<<tiempo_total.count()<<"
microsegundos."<<std::endl;
    return 0;
}
```







Tiempo de ejecución: 195.1230 microsegundos.

Las tareas de los tres hilos se distribuyeron siguiendo la arquitectura MISD, de modo que cada hilo procesa los mismos 100 registros del dataset, pero realiza una operación diferente. El primer hilo calcula el consumo eléctrico, el segundo determina el costo y el tercero obtiene el promedio diario, almacenando cada resultado en un vector independiente. Para ello, se utilizaron funciones lambda y `std::thread`, junto con `join()` para esperar a que los tres hilos finalizan. Además, se corrigió la división del promedio diario mediante la

conversión a tipo `double` (multiplicación del 1.0), evitando la pérdida de decimales por la división entera.